MediDispense: setup and user guide

Install it on a new computer, build the conveyor, log in, and run a delivery from prescription to bedside confirmation.

What the system is. A doctor's prescription creates a delivery job. A small ESP32-driven conveyor carries the medicine box to Bed 1 or Bed 2, a servo pushes it into that bed's chute, and a nurse confirms it at the bedside. Everything is stored in a database and shown live in a web app.
Four repositories make up the project (all under **github.com/SHIVAAKARTHIK**): `conveyor-docs` (this guide, report, wiring), `conveyor-backend` (API and database), `conveyor-admin-web` (web app), and `conveyor-firmware-` (ESP32 code). Note the trailing hyphen in the last name.

1. What you need

Software	Version tested	Why
Git	any recent	Download the four repositories
Python	3.13 (3.11 or newer should work)	Backend
Node.js	22 (20 or newer)	Web app
arduino-cli	1.5.1	Build and flash the ESP32 (the Arduino IDE 2 also works)
ESP32 board support	esp32 core 3.3.11	Installed with arduino-cli (step 7)
USB driver for the board	Silicon Labs CP210x (or CH340)	So the ESP32 appears as a COM port
Chrome or Edge	current	Web app

Hardware (conveyor)	Notes
ESP32 DevKit V1	30-pin, with a USB cable that carries data (not charge-only)
L298N motor driver	Remove its **ENA jumper**
Conveyor belt kit with DC gear motor	About 20 cm belt, 6.5 cm wide
2 x SG90 servos	One pusher per bed, on opposite sides of the belt
2 x IR obstacle sensors	One per bed
Active buzzer module, push button	
12 V 2 A DC adapter	Powers the L298N and the belt motor only
Jumper wires, a small test box	Box about 4 x 3 x 1.5 cm

The computer and the conveyor must be on the **same 2.4 GHz WiFi** network. The ESP32 cannot use 5 GHz, and a guest network or "AP isolation" setting on the router will stop the board reaching the computer.

2. Set it up on a new computer

Step 1. Download the four repositories

```
git clone https://github.com/SHIVAAKARTHIK/conveyor-backend.git
git clone https://github.com/SHIVAAKARTHIK/conveyor-admin-web.git
git clone https://github.com/SHIVAAKARTHIK/conveyor-firmware-.git
git clone https://github.com/SHIVAAKARTHIK/conveyor-docs.git
```

Step 2. Start the backend

In PowerShell, inside the `conveyor-backend` folder:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
copy .env.example .env
.\.venv\Scripts\python.exe seed_demo.py
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

- `seed_demo.py` creates the database `dev.db` and adds all the demo accounts from section 3 (admin, eight doctors, two nurses, a technician and the conveyor's login). It is safe to run again.
- Prefer no script? `copy demo.db dev.db` gives the same data ready made. Do this before the first start.
- **Keep `--host 0.0.0.0`.** Without it only this computer can reach the backend, and the conveyor cannot.
- Check: open **http://localhost:8000/docs**. You should see the API page.

Step 3. Allow the conveyor through the Windows firewall

The ESP32 calls the backend on port 8000. The first time Python runs, Windows may ask to allow it: choose **Private networks**. Or add the rule once, in PowerShell opened **as Administrator**:

```
New-NetFirewallRule -DisplayName "MediDispense backend" -Direction Inbound -Protocol TCP -LocalPort
8000 -Action Allow -Profile Private
```

Your WiFi must be set to a **Private** network in Windows for this rule to apply.

Step 4. Start the web app

In a second terminal, inside the `conveyor-admin-web` folder:

```
npm install
copy .env.example .env.local
npm run dev
```

Open **http://localhost:3000**. Leave both terminals running.

Step 5. Log in

Use any account from section 3, for example the admin. If you can sign in and see the dashboard, the backend and the web app are working together.

Step 6. Wire the conveyor

Wire it as in section 5, **with the USB unplugged**. Wiring while powered made the board restart during testing. Plug the USB back in when you are done.

Step 7. Prepare and flash the conveyor firmware

In `conveyor-firmware-`, copy the example settings and edit them:

```
copy config.h.example config.h
```

Setting in `config.h`	What to put
`WIFI_SSID`, `WIFI_PASSWORD`	Your 2.4 GHz WiFi name and password
`BACKEND_HOST`	This computer's name. Run `hostname` in PowerShell. The board finds the computer as `name.local`.
`BACKEND_URL`	Fallback if the name lookup fails: `http://` + this computer's IP (`ipconfig`, "IPv4 Address") + `:8000`
`STATION_EMAIL`, `STATION_PASSWORD`	The conveyor's own login from section 3 (`conveyor.station@medidispense.com`)

config.h holds your WiFi password. Git ignores it. Never commit or share it. Then install the board support and the two libraries once, and flash:

```
arduino-cli core update-index --additional-urls https://espressif.github.io/arduino-esp32/package_esp32_index.json
arduino-cli core install esp32:esp32@3.3.11 --additional-urls https://espressif.github.io/arduino-esp32/package_esp32_index.json
arduino-cli lib install "ArduinoJson@7.4.3" "ESP32Servo@3.2.1"
arduino-cli board list
arduino-cli compile --fqbn esp32:esp32:esp32 .
arduino-cli upload -p COM3 --fqbn esp32:esp32:esp32 .
```

Use the COM port that `board list` shows instead of COM3. If the upload hangs at "Connecting...", hold the board's **BOOT** button until it starts.

Step 8. Check that everything talks

Check	Expected
Open **http://localhost:8000/docs**	API page loads
Open **http://conveyor.local/status**	A line of settings (belt, sensors, mode "delivery")
Sign in as the technician: **Live hardware control**	"Conveyor online"
Click **Beep**, then **Push Bed 1** and **Push Bed 2**	The buzzer sounds and each servo swings out and back
Click **Start belt**, then **Stop belt**	The belt moves, then stops

If all of those work, go to section 4.

3. Logins

> **Demo passwords.** These accounts come with the demo database and the passwords are public. They are fine on your own computer for a project demo. The web app has no "change password" screen yet, so to change one, delete the account on the admin **Staff** page and create it again with a new password. Do this before anyone outside your team uses the system.

Role	Name	Email	Password
Admin	Default Admin	`admin@medidispense.com`	`admin123`
Nurse	Nurse Priya	`nurse@medidispense.com`	`pass123`
Nurse	Nurse Ramesh	`nurse2@medidispense.com`	`pass123`
Technician	Technician Suresh	`technician@medidispense.com`	`pass123`
Conveyor (machine)	Conveyor Station	`conveyor.station@medidispense.com`	`station123`

Doctors (password `pass123` for all)

Speciality	Doctor	Email
Cardiology	Dr. Arjun Rao	`dr.arjun.rao@medidispense.com`
	Dr. Meera Nair	`dr.meera.nair@medidispense.com`
Gastroenterology	Dr. Kavya Shetty	`dr.kavya.shetty@medidispense.com`
	Dr. Rohit Verma	`dr.rohit.verma@medidispense.com`
Nephrology	Dr. Anita Desai	`dr.anita.desai@medidispense.com`
	Dr. Sanjay Kulkarni	`dr.sanjay.kulkarni@medidispense.com`
Pulmonology	Dr. Farhan Ali	`dr.farhan.ali@medidispense.com`
	Dr. Lakshmi Iyer	`dr.lakshmi.iyer@medidispense.com`

Patients have no preset logins. Each patient registers with **Sign up** and chooses their own email and password. The conveyor's login must match `STATION_EMAIL` and `STATION_PASSWORD` in `config.h`. A new doctor needs a **department** and must be **Available** before patients can find them.

4. Create and run a delivery job

> **The key idea.** The **doctor's prescription creates the job** automatically. The technician does not create it; the technician loads the box and starts the belt. A **nurse assigns the bed** first, because the bed decides the route: **Bed 1** uses sensor A and servo 1; **Bed 2** uses sensor B and servo 2.

1) Patient registers (no bed yet)

1. On the login page click **Sign up**. Fill in name, age, gender, optional phone, email and password.
2. Click **Check in**. The home page says "Waiting for a nurse to assign your bed."

2) Nurse assigns the bed

1. Log out and sign in as a **nurse**. You land on **Bedside confirmation**.
2. In **Patients and beds** the new patient is highlighted at the top. Choose **Bed 1** or **Bed 2** and click **Assign**.

3) Patient asks a doctor

1. Sign in as the patient. Click **Find a Doctor**, choose a speciality, and pick a doctor who shows **Available**.

2. Describe the problem and click **Send request**.

4) Doctor accepts and prescribes (this creates the job)

1. Sign in as that doctor. The green **Available** button at the top right must not say **Busy**.
2. Under **Requested** click **Accept**. Under **Accepted: ready to prescribe** click **Prescribe**.
3. Fill in medicine, dosage, form, quantity and notes (**+ Add another medicine** for more). Click **Send prescription**.
4. A delivery now appears as `preparing`. If the patient has no bed you will see "ask a nurse to assign one before prescribing": go back to step 2.

5) Technician starts the belt

1. Sign in as the **technician**. **Live hardware control** must say **Conveyor online**.
2. The board picks the job up within 4 seconds and beeps. The panel reads "A box for Bed N is waiting."
3. **Put the box on the belt before sensor A.** Keep your hands clear of the servo arms.
4. Press the machine's **push button once**, or click **Dispatch Bed N box** on the page.

What happens next, by itself: the belt starts; the sensor at that bed sees the box and stops the belt (a Bed 2 box rides past sensor A, which is ignored); that bed's servo pushes the box into the chute; the buzzer gives one long beep; the delivery changes to `arrived`.

Press only once. A second press during the run is the **emergency stop** and marks the delivery failed. A run also fails if the box does not reach its sensor within **40 seconds**. For the next job nothing needs flashing: when another prescription is waiting, press the button again and it starts that job.

6) Nurse confirms at the bedside

1. Sign in as a nurse. Under **Ready to confirm** find the delivery marked `arrived`.
2. Enter the patient's wristband code (a barcode scanner types it for you). Without a scanner click **Fill from record (demo)**.
3. Click **Confirm delivery**. A wrong code is rejected. The delivery becomes `confirmed` and the prescription `delivered`.

7) Patient sees the result

The patient's home page shows **Delivered**. If the nurse has not confirmed yet, the patient can click **I received it**.

5. Conveyor wiring

ESP32 DevKit V1, 30-pin. Wire with the USB unplugged.

Device	Wire	ESP32 pin
Sensor A (Bed 1)	S, orange	**D4**
Sensor B (Bed 2)	S, orange	**D32**
Servo 1 (Bed 1, near side)	signal, yellow	**D18**
Servo 2 (Bed 2, far side)	signal, yellow	**D19**
Buzzer	SIG, black	**D23**
Push button	one leg (other leg to ground one)	**D15**
L298N ENA (speed)		**D25**
L298N IN1, IN2 (direction)		**D26**, **D27**

Power and ground rails

Rail	ESP32 pin	Wires joined on it
Ground one	one GND pin (beside VIN)	Both sensors' G (green), button's other leg, buzzer GND (yellow)
3V3	3V3	Both sensors' V (brown)
Ground two	the other GND pin (beside 3V3)	Both servos' ground (brown), L298N GND
VIN (5 V)	VIN (not VN)	Both servos' power (red), buzzer VCC (white)

Motor side (L298N)	Goes to
Belt motor's two wires	Output terminals OUT1 and OUT2 (swap them if the belt runs the wrong way)
12 V adapter +	L298N **12V** terminal
12 V adapter -	L298N **GND** terminal, also joined to ground two
ENA jumper	Removed. The L298N 5V terminal is not connected.

Brown is ground on the servos but 3V3 power on the sensors, so keep those rails apart. A sensor's V wire must end on 3V3, never on a GND pin. The buzzer's red NC wire is unused. The 12 V adapter powers only the L298N; the ESP32 runs from USB. The mechanical layout (servos on opposite sides, sensors just before each servo's sweep) and the full wiring diagram are in the report in `conveyor-docs`.

6. Changing the WiFi, or using another computer

New WiFi

1. Connect the board by USB. The computer must be on the **new** WiFi too.
2. Edit `WIFI_SSID` and `WIFI_PASSWORD` in `config.h` (2.4 GHz only).
3. Compile and upload (step 7 of section 2). After about 15 seconds the board is back on `http://conveyor.local`.

New computer running the backend

Repeat steps 2 to 4 of section 2 on that computer, then in `config.h` set `BACKEND_HOST` to its name and `BACKEND_URL` to its IP, and flash again. If the new computer opens the web app from another device, add that address to `CORS_ORIGINS` in the backend's `.env`.

The computer's IP address can change when the router restarts. The board looks the computer up by name every minute, so this is usually fine. If it is not, update `BACKEND_URL` or give the computer a fixed IP in the router.

7. If something goes wrong

Problem	Check
Login fails	Did you run `seed_demo.py` (or copy `demo.db`)? Is the backend running? Check `.env` exists.
Web page cannot reach the backend	Backend running on port 8000? `.env.local` points at `http://localhost:8000`?
The board never picks up a job	Backend started with `--host 0.0.0.0`? Firewall rule added? Same WiFi? Station email and password in `config.h` match an account? The board's serial monitor (115200) shows "logged in to the backend".
"Conveyor not reachable" on the page	Board powered and on the same WiFi as this computer? If `conveyor.local` does not work (VPN, some routers), use the board's IP in the **Conveyor address** box.
Doctor not listed for the patient	The doctor must be **Available** and have a department.
Doctor cannot prescribe	The patient has no bed yet. A nurse must assign one.
Button or Dispatch does nothing	No job is waiting. Check a prescription exists and the page shows `preparing`.
Belt does not move	L298N power LED lit? Motor wires tight on OUT1 and OUT2? ENA, IN1, IN2 wires on D25, D26, D27? ENA jumper removed?
Box never stops at the sensor	The box must start before the sensor and pass close to its face. Check the sensor LED lights when you pass a hand in front.
Servo does not move	Signal wire on D18 or D19, red on VIN, brown on ground two. Test with **Push Bed 1** and **Push Bed 2** on the page.
No buzzer sound	White VCC to VIN rail, yellow GND to ground one, black SIG to D23. Test with **Beep**.
Run ended as "failed"	A second button press (emergency stop), or 40 seconds passed without the box reaching its sensor.
Delivery stuck in progress	On the technician page click **Mark failed**.
Board restarts on its own	A loose USB cable or wire, or a wire plugged in while powered. Unplug, re-seat, plug in again.
Start again with clean data	Stop the backend, delete `dev.db`, then run `seed_demo.py` again (or copy `demo.db` over it).

8. Before real use

- Change the demo passwords (delete and recreate each account on the **Staff** page).
- Set a long random `JWT_SECRET_KEY` in the backend's `.env`.
- The conveyor's local control page uses plain HTTP and has no password. Keep it on a private network.
- This is a model-scale prototype for two beds and one box at a time. It is not a clinical device and has not been validated for real medicines.